1. Interconnection Networks for Parallel Processors

Interconnection networks let all processors in a parallel system talk to each other efficiently — they are the "roads" that connect processing units.

Two Main Approaches

Type	Meaning
Static Interconnection Network	Fixed connections between nodes; path doesn't change
Dynamic Interconnection Network	Connections can be reconfigured on the fly

💡 ***Memory tip:*** *Static = Stuck in place. Dynamic = can Dance/change around.*

A. Static Interconnection Network

- **Unidirectional:** data flows one way only (A → B, not B → A)
- **Bidirectional:** data flows both ways (A ↔ B)

(i) Fully / Completely Connected Network

Every node is directly connected to every other node — maximum connectivity.

$$\text{Links} = n(n-1) / 2$$

Each node connects to (n−1) other nodes

Example: If n = 6 → Links = 6×5/2 = 15, and each node connects to 5 others.

Feature	Detail
Connectivity	Highest possible — direct link between any pair
Bandwidth / Latency	Best case: low latency, high bandwidth
Cost & Complexity	Very expensive — needs the most wires
Data Transmission	Fast & direct (no intermediate hops)

(ii) Limited Connection Network (Partial Network)

Each node connects to only a subset of nodes (not all). Cheaper but less powerful than fully connected.

Feature	Fully Connected	Limited Connection
Connectivity	Every node ↔ every node	Fewer links per node
Cost	High	Low
Bandwidth	High	Limited
Fault Tolerance	High	Lower
Scalability	Poor (grows fast)	Good

Common examples: Ring, Tree, Linear Array, Mesh, Hypercube networks.

Linear Network

- Nodes connected in a single row/line, one after another.
- Weakness: if one node fails, every node after it stops working (chain breaks).

Ring Network

- Exactly like a linear network, but the last node is also connected back to the first node — forming a loop.

Three Cube (Hypercube) Network

- 3 nodes/switches interconnected in a cube-shaped pattern for redundancy and fault tolerance.
- Used in data-center design where backup paths matter.

Two-Dimensional (Mesh) Network

- Nodes arranged in a grid (rows × columns), each node linked to its neighbors.
- Common in distributed computing for structured, predictable communication.

B. Dynamic Interconnection Network

Unlike static networks, connections are NOT fixed — they reconfigure themselves as communication needs change.

★ 6 Key Aspects of Dynamic Networks

- Reconfiguration — paths can change
- Switching Mechanisms — how paths are set up
- Flexibility — adapts to different traffic patterns
- Scalability — handles growth well
- Fault Tolerance — can reroute around failures
- Communication Efficiency — optimized data flow

2. Code Optimization for Multi-Core CPUs & GPUs

Optimization = making software use hardware (cores, memory, cache) as efficiently as possible, since raw hardware speed alone isn't enough.

Goals of Optimization: Improve execution speed & maximize throughput, Reduce memory latency & communication overhead, Minimize energy consumption, Balanced workload — no core idle while others overload, Better cache utilization & scalability

4 Types of Parallelism

Type	What It Means
Instruction-Level (ILP)	Multiple instructions run together inside one CPU pipeline
Thread-Level (TLP)	Multiple threads run concurrently on multi-core CPUs
Data-Level (DLP)	Same operation applied to many data items at once (GPU/vector)
Task-Level	Independent tasks spread across different processing units

Optimization on Multi-Core CPUs

- **Thread Parallelism:** split a big task into smaller threads. Too many threads = overhead → solved via thread pooling.
- **Load Balancing:** even distribution of work; dynamic scheduling beats static assignment.
- **Memory Optimization:** improve cache locality.
  - • Temporal locality — reuse recently accessed data
  - • Spatial locality — access nearby memory locations together
  - • Techniques: loop tiling, loop fusion, loop interchange
- **False Sharing:** multiple threads modify different variables that sit on the SAME cache line → unnecessary invalidation. Fix: padding/aligning data.
- **Synchronization:** locks/semaphores/barriers are needed but slow things down — minimize critical sections; prefer lock-free/atomic operations.

Optimization on GPUs

GPUs use massive parallelism: thousands of lightweight threads, organized as Grid → Blocks → Warps → Threads.

Concept	Explanation
Warp	Group of 32 threads executing the same instruction together
Warp Divergence	Threads in a warp take different branches → serialized (slow) execution
Memory Coalescing	Adjacent threads access adjacent memory → combined into fewer transactions
Shared Memory	Fast memory for threads within a block; watch out for bank conflicts
Occupancy	How many warps are active at once; higher = better latency hiding
Data Transfer (CPU↔GPU)	Major bottleneck; use pinned memory + async streams to overlap
Kernel Fusion	Combine multiple GPU kernels into one → less launch overhead

3. Routing & Switching in Parallel/Distributed Systems

- **Switching:** HOW data physically moves from source to destination through intermediate nodes.
- **Routing:** WHICH path/route the data takes.

Types of Switching

Type	How It Works	Trade-off
Circuit Switching	Dedicated path reserved before transfer begins	Predictable, but wastes resources on bursty traffic
Packet Switching	Data split into packets, each may take a different route	Efficient sharing, but variable delay + reordering needed
Message Switching	Whole message stored & forwarded at each node (no split)	No dedicated path, but high latency (store-and-forward delay)

Modern Switching Techniques

Technique	Behavior
Store-and-Forward	Waits for the ENTIRE packet before forwarding — reliable but slow
Cut-Through	Forwards as soon as the header is read — lower latency
Wormhole	Splits packets into small “flits”, pipelines them through network — fastest, used in HPC

Routing

Objectives: minimize delay, reduce congestion, ensure fault tolerance, maximize throughput, and stay scalable.

Routing Type	Description	Trade-off
Deterministic	Fixed path, always the same (e.g., XY routing in mesh)	Simple & predictable, but can't adapt to congestion/failure
Adaptive	Path changes based on live network conditions	Better performance/fault tolerance, but complex hardware
Oblivious	Path chosen randomly / by fixed probability, ignoring conditions	Simple, but not optimal under heavy traffic

Deadlock in Routing

Deadlock = packets permanently blocked, each waiting on a resource held by another — nobody can move.

- Prevention techniques: virtual channels, resource ordering, deadlock-free routing algorithms.

Routing in NoC (Network-on-Chip) vs Distributed Systems

- **NoC:** connects cores inside a single chip; limited area/power → needs efficient deterministic/adaptive routing + virtual channels.
- **Distributed Systems:** routing over clusters/cloud; e.g. MPI handles routing decisions; cloud systems also load-balance across servers.

Performance Metrics

Metric	Meaning
Latency	Time for data to travel from source → destination
Throughput	Amount of data successfully transmitted per unit time

Trade-off: Deterministic routing = simple but less flexible. Adaptive routing = better performance but more complex.

4. Butterfly Network

A multi-stage interconnection network shaped like butterfly wings — scalable, predictable, and efficient for high-performance computing.

5 Core Principles

★ Remember: F-L-P-S-D

- Fixed Paths — predefined routes, avoids conflicts
- Logarithmic Distance — hops grow as log(n), keeps latency low
- Parallelism — supports many simultaneous transfers
- Scalability — add nodes/stages without redesign
- Deterministic Routing — fixed, reliable delivery paths

Structure

- Multiple stages; each stage's nodes connect to nodes in the next stage.
- Each node links to 2 nodes in the next stage → every input can reach every output.

- Number of stages grows logarithmically with number of inputs.
- Symmetrical structure (like butterfly wings) — simplifies routing.
- Multiple/redundant paths exist → improves fault tolerance.

Routing Algorithms

Algorithm	Idea
Deterministic	Fixed route per source-destination pair
Adaptive	Adjusts based on congestion/traffic
Minimal	Shortest path (fewest hops)
Non-Minimal	Longer path to avoid congested nodes
Fault-Tolerant	Uses redundant paths if a node/link fails

Communication Patterns

Pattern	Description
One-to-One	Direct transfer between two nodes
One-to-All	Broadcast — one node sends to everyone
All-to-One	Aggregation — all nodes send to one (e.g. gathering results)
All-to-All	Every node sends to every other node
Many-to-Many	Groups communicate with other groups
Pipeline	Data passed through a sequence of nodes, stage by stage

Advantages vs Challenges

Advantages	Challenges
High efficiency, low congestion	Complex to design & implement
Scalable (add nodes easily)	High initial hardware cost
Low latency (log distance)	Difficult maintenance (many links)
Parallel processing support	Fault management is complex
Predictable performance	Limited flexibility (fixed paths)
Fault tolerant (redundant paths)	Scalability issues at very large size

Use Cases: High-Performance Computing (supercomputers), Data Centers, Scientific Simulations, Big Data Analytics, Real-Time Applications (gaming/trading), Telecommunications, Cloud Computing.

5. Z-Transformation in Distributed Systems

A math tool (from signal processing) used to analyze systems that change in discrete time steps — like the Laplace transform, but for discrete (step-by-step) systems instead of continuous ones.

$$X(z) = \Sigma\, x[n] \cdot z^{-n} \quad (n = 0 \text{ to } \infty)$$

Converts a discrete sequence x[n] into frequency-domain form

Why Distributed Systems Need It

Distributed systems are discrete-event systems: messages arrive at intervals, clocks tick discretely, retries happen after timeouts, queues change step-by-step, consensus happens in rounds → all modeled as difference equations → perfect for Z-transform analysis.

Main Uses

1. Network Delay & Congestion Analysis

$$q[n+1] = q[n] + a[n] - d[n]$$

q=queue length, a=arriving packets, d=departing packets

- Used for: TCP congestion control, router buffer optimization, traffic shaping.

2. Distributed Control Systems

- Used in distributed databases & load balancing (e.g., cloud auto-scaling).
- Z-transform reveals: whether scaling oscillates, convergence speed, stability under delay.

3. Clock Synchronization

- Used in protocols like NTP, PTP, and sensor networks — which use recursive clock correction algorithms.

4. Reliability & Failure Recovery

- Retry mechanisms are modeled as discrete-time recursive systems.

6. Cloud Computing

Delivery of computing services (servers, storage, databases, networking, software) over the internet — no need to own physical hardware.

NIST Definition

"A model for enabling convenient, on-demand network access to a shared pool of configurable computing resources."

5 Characteristics

Characteristic	Meaning
On-Demand Self-Service	Access resources anytime, without human intervention
Broad Network Access	Available over the internet from any device
Resource Pooling	Shared resources serve multiple users
Rapid Elasticity	Scale up/down quickly
Measured Service	Pay only for what you use

Deployment Models

Model	Owner	Key Trait
Public Cloud	Third-party provider (AWS, GCP, Azure)	Cheap, scalable, shared — less control/security
Private Cloud	Single organization	High control & security, but expensive
Hybrid Cloud	Mix of public + private	Flexible, cost-optimized, but complex to manage
Community Cloud	Shared by orgs with similar needs	e.g., universities, govt departments

Service Models

Model	Provides	Examples	Benefit
IaaS	Virtual machines, storage, networks	AWS EC2, Azure VMs	Full infrastructure control
PaaS	Platform to build/deploy apps	Google App Engine, Azure App Services	Faster development
SaaS	Ready-to-use software	Google Workspace, Microsoft 365	No installation, auto-updates

Major Public Cloud Platforms

Platform	Key Services	Strength
AWS	EC2, S3, RDS, Lambda	Largest service portfolio, global infrastructure
Google Cloud (GCP)	Compute Engine, BigQuery, Kubernetes Engine	Strong AI/analytics + containers
Microsoft Azure	Virtual Machines, SQL DB, Active Directory	Best Microsoft product integration

Resource Management Functions

Function	Description
Resource Allocation	Assigning CPU, memory, storage
Load Balancing	Distributing workloads evenly
Virtualization	Creating virtual resources
Auto Scaling	Automatically increasing/decreasing resources
Monitoring	Tracking system performance

Security

Risks	Measures
Data breaches	Encryption
Unauthorized access	Authentication
Malware attacks	Firewalls
Data loss	Backup & Recovery
Insider threats	Access Control

★ Shared Responsibility Model

- Cloud provider secures the infrastructure
- Customer secures their own applications & data

7. Algorithms for Systems of Linear Equations

A system of linear equations is a set of equations sharing the same variables. Important in engineering, ML, graphics, and scientific computing.

$$AX = B$$

A = coefficient matrix, X = unknowns, B = constants

Gaussian Elimination

- Step 1: Select pivot element
- Step 2: Eliminate lower entries
- Step 3: Repeat until upper-triangular matrix formed
- Step 4: Apply back substitution

Advantages	Disadvantages
Accurate for small systems	Computationally expensive for large matrices
Deterministic (exact) solution	Poor scalability in sequential systems

LU Decomposition

$$A = L \times U$$

L = lower triangular matrix, U = upper triangular matrix

- Benefit: efficient when solving multiple systems that share the same coefficient matrix A.
- Applications: scientific computing, numerical simulations, optimization problems.

Parallel Gaussian Elimination

What's Parallelized	Benefits	Challenges
Row operations, matrix partitioning, elimination	Reduced execution time, better hardware use, suits large matrices	Processor sync, communication overhead, load balancing

- **Distributed Memory Systems:** matrix blocks distributed among nodes; nodes communicate via message passing (MPI — Message Passing Interface).

8. RFID (Radio Frequency Identification) — IoT

A wireless technology for automatic identification and tracking of objects using radio waves. A core building block of IoT.

Components

Component	Role
RFID Tag	Stores object info (Passive / Active / Semi-passive types)
RFID Reader	Reads data from tags
Antenna	Transmits radio signals between reader and tag
Database/System	Stores & processes collected data

Working Principle (4 Steps)

- 1. Reader sends a radio signal
- 2. RFID tag receives the signal
- 3. Tag transmits its stored data back
- 4. Reader receives data and forwards it to the database

Applications

Sector	Examples
Supply Chain	Inventory tracking, shipment monitoring, warehouse automation
Healthcare	Patient tracking, medicine ID, hospital asset management
Smart Transportation	Toll collection, vehicle tracking, smart parking
Retail	Automated billing, product tracking, theft prevention
Smart Agriculture	Livestock tracking, crop monitoring, equipment management

Advantages vs Challenges

Advantages	Challenges
Fast identification	Security concerns
Contactless communication	Privacy issues
Real-time tracking	High implementation cost
Increased automation	Signal interference
Reduced human effort	Limited reading range

9. Strassen's Matrix Multiplication

A divide-and-conquer algorithm that multiplies matrices FASTER than the traditional method by trading multiplications for (cheaper) additions.

Traditional vs Strassen

	Traditional	Strassen's
Multiplications (2×2)	8	7
Complexity	$O(n^3)$	$O(n^{2i}, ^1)$

Memory tip: 8 → 7: Strassen trades ONE multiplication for extra additions/subtractions — and additions are cheap!

The 7 Products (M1–M7)

```
M1 = (A11 + A22)(B11 + B22)
M2 = (A21 + A22) B11
M3 = A11 (B12 – B22)
M4 = A22 (B21 – B11)
M5 = (A11 + A12) B22
M6 = (A21 – A11)(B11 + B12)
M7 = (A12 – A22)(B21 + B22)
```

Result Sub-matrices

```
C11 = M1 + M4 – M5 + M7
C12 = M3 + M5
C21 = M2 + M4
C22 = M1 – M2 + M3 + M6
```

Pros & Cons

Advantages	Disadvantages
Faster for large matrices	Complex implementation
Suitable for parallel processing	More memory usage
	Less efficient for very small matrices

Parallel Execution Model

Since M1–M7 are all independent, they can run in parallel:

Model	Approach
OpenMP (shared memory)	#pragma omp parallel sections — each thread computes one Mi (Thread 1→M1, ... Thread 7→M7)
MPI (distributed memory)	Master process sends A/B partitions to worker processes; each worker computes one Mi and sends results back

After all M1–M7 are computed, the master/root combines them into result matrix C.

10. Iterative Methods for Linear Systems

Instead of solving exactly (like Gaussian Elimination), iterative methods start with a guess and keep improving it — great for large, sparse matrices.

Jacobi Method

Uses OLD values from the previous iteration to compute ALL new values at once.

Advantages	Disadvantages
Simple to implement	Slow convergence
Easy to parallelize — great for distributed systems	Requires a diagonally dominant matrix

Gauss-Seidel Method

Uses NEWLY computed values immediately (within the same iteration) — unlike Jacobi.

Advantages	Disadvantages
Faster convergence than Jacobi	Harder to parallelize
Better accuracy	Sequential dependency exists

Side-by-Side Comparison

Aspect	Jacobi	Gauss-Seidel
Uses values from	Previous iteration only	Current iteration (as soon as available)
Convergence speed	Slower	Faster

Aspect	Jacobi	Gauss-Seidel
Parallelization	Easy (independent updates)	Difficult (depends on updated values)
Best fit	GPU / HPC / distributed systems	Single-processor / sequential systems

Convergence

$$||X^{(k+1)} - X^{(k)}|| < \varepsilon$$

ε = small tolerance value; stop when change is smaller than ε

- Conditions: matrix must be diagonally dominant, and spectral radius must be less than 1.

Parallelizing Gauss-Seidel

- Since it's naturally sequential, solutions include: block decomposition, domain decomposition, and hybrid methods.

11. OpenMP Quick Reference

OpenMP is a shared-memory API that lets you parallelize C/C++ code using simple #pragma directives.

Key Directives

Directive	Purpose
#pragma omp parallel	Creates a team of threads that run the block together
#pragma omp parallel for	Splits a for-loop's iterations across threads
private(var)	Each thread gets its own private copy of a variable
reduction(+:sum)	Safely combines a variable (e.g. sum) across all threads
omp_get_thread_num()	Returns the calling thread's ID number

Example: Parallel Sum with Reduction

```
#include <stdio.h>
#include <omp.h>

int main() {
    int i, sum = 0;
    int arr[10] = {1,2,3,4,5,6,7,8,9,10};

    #pragma omp parallel for reduction(+:sum)
    for (i = 0; i < 10; i++) {
        sum = sum + arr[i];
    }

    printf("Total Sum = %d\n", sum);   // Output: 55
    return 0;
}
```

Memory tip: *reduction(+:sum) avoids race conditions — each thread keeps a private sum, then OpenMP adds them all together at the end.*

12. Applications in Parallel & Distributed Systems

Domain	Examples
Scientific Computing	Weather prediction, fluid dynamics, molecular simulations
Machine Learning	Optimization algorithms, neural network training
Big Data Analytics	Distributed processing frameworks, parallel matrix operations
Computer Graphics	Image processing, 3D rendering, simulation systems
IoT & Smart Systems	RFID tracking, sensor data processing, distributed monitoring